

Hackathon Integrado IADT + SOAT — Entrega Final

FIAP Secure Systems — Análise de Diagramas de Arquitetura com IA

Campo	Valor
Aluno	Daniel Roberto Pereira
RM	365742
Discord	danielroberto5075
Curso	POSTECH SOAT (Software Architecture)
Data de entrega	Maior2026

Repositórios GitHub

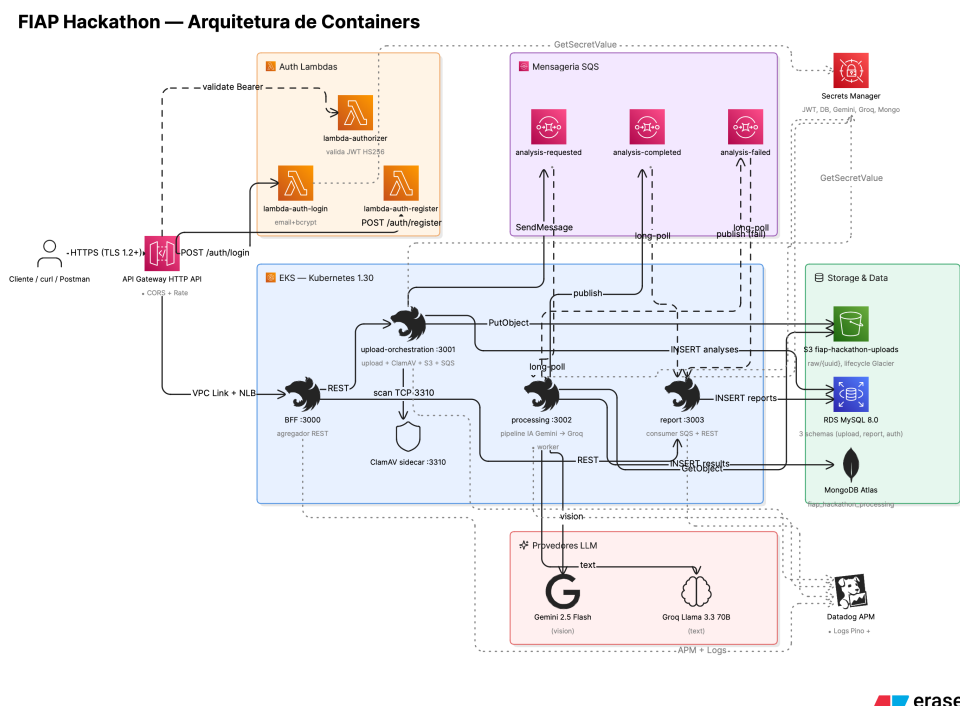
Total: **6 repositórios** no GitHub (organização DanielRoberto72). O usuário soat-architecture foi adicionado como colaborador em todos eles. O repositório fiap-hackathon-lambda-auth é público para permitir que o bootstrap do LocalStack baixe o asset do GitHub Release.

#	Repositório	Descrição	Link
1	fiap-hackathon-infra	Terraform + Helm + LocalStack init + docker-compose E2E + docs + diagramas	github.com/DanielRoberto72/fiap-hackathon-infra
2	fiap-hackathon-bff	BFF NestJS agregador (porta 3000) — agrega upload + report e expõe REST público	github.com/DanielRoberto72/fiap-hackathon-bff
3	fiap-hackathon-upload-orchestration	Upload + validação MIME/magic-bytes + ClamAV + S3 + SQS analysis-requested (porta 3001)	github.com/DanielRoberto72/fiap-hackathon-upload-orchestration
4	fiap-hackathon-processing	Worker SQS — pipeline IA Gemini Vision + Groq Llama 3.3 70B + Mongo (porta 3002)	github.com/DanielRoberto72/fiap-hackathon-processing
5	fiap-hackathon-report	Consumer SQS analysis-completed/failed + REST GET /api/reports/{id} (porta 3003)	github.com/DanielRoberto72/fiap-hackathon-report
6	fiap-hackathon-lambda-auth	Lambda authorizer + login + register (HS256 JWT). Zip empacotado em GitHub Release	github.com/DanielRoberto72/fiap-hackathon-lambda-auth

Vídeo de Demonstração

Link do Vídeo: <https://youtu.be/J43rU2ctHPs>

Arquitetura



Plataforma de microsserviços NestJS expostos por um API Gateway HTTP API com JWT Lambda authorizer. Cliente envia diagramas para o BFF; o upload-orchestration valida e persiste no S3; o processing consome a fila SQS, roda o pipeline IA em duas etapas (Gemini Vision + Groq Llama 3.3) e publica analysis-completed; o report materializa cópia local em MySQL e expõe consulta REST. Em ambiente local os serviços AWS são emulados via LocalStack (S3, SQS, SNS, Secrets Manager, Lambda, API Gateway, IAM).

1. Arquitetura de Microsserviços e Comunicação

1.1 Estratégia escolhida

Adotamos **Event-Carried State Transfer (ECST) via SQS** como base da comunicação assíncrona. Cada microsserviço é dono do seu banco e publica eventos com payload completo no momento em que termina seu trabalho local. Os consumidores reagem aos eventos de forma autônoma, sem chamar de volta o produtor. A comunicação síncrona (REST) é restrita ao caminho cliente → API Gateway → BFF → upload.

1.2 Justificativa da escolha

- **Desacoplamento total:** processing não conhece report (publica analysis-completed e segue).
- **Sem orquestrador central:** não há single point of failure no fluxo principal.
- **Escalabilidade independente:** processing pode escalar workers conforme tamanho da fila; report continua leve.
- **Resiliência embutida:** visibility timeout + 3 retries automáticos + Dead Letter Queue + alerta Datadog quando esgota.
- **Aderência ao requisito do PDF:** microsserviços com pelo menos um fluxo assíncrono.

1.3 Implementação técnica

Componente	Função
AWS SQS analysis-requested	Upload publica → Processing consome (long-poll, batch size 5)
AWS SQS analysis-completed	Processing publica payload completo → Report consome e materializa
AWS SQS analysis-failed	Processing publica em falha permanente → Report registra como FAILED
Dead Letter Queues (2x)	analysis-requested-dlq + analysis-completed-dlq após 3 tentativas
Idempotency-Key	Upload exige header — POSTs duplicados devolvem o mesmo analysisId

1.4 Fluxo de compensação e degradação

Em vez de Saga completa (que faz pouco sentido no domínio análise-de-diagrama), aplicamos **graceful degradation**: se Gemini Vision falhar 3x, o pipeline troca para MockLlmProvider (saída determinística) e publica analysis.completed com degraded=true, sinalizando ao consumidor que o relatório precisa de revisão humana. Falhas não-retentáveis (Zod schema inválido, S3 GET 404) vão direto para analysis.failed com reason enumerado.

2. Divisão dos Microsserviços (Bounded Contexts)

2.1 Critérios de decomposição

Aplicamos o princípio de **Bounded Contexts do DDD + polyglot persistence**. Cada serviço encapsula um domínio coeso, é dono do seu banco e expõe apenas o necessário. Nenhum serviço lê o banco do outro — a integração é por evento.

2.2 BFF (porta 3000)

Domínio: Agregação síncrona REST + roteamento.

- Responsabilidades: agregar upload + report sob uma única API pública.
- Sem persistência — atua como pass-through autenticado.
- Stack: NestJS 11 + TypeScript + Pino + Axios.

2.3 Upload-orchestration (porta 3001)

Domínio: Receber diagrama, validar, persistir no S3, disparar pipeline.

- Validação: MIME + magic bytes (file-type) + tamanho máximo + ClamAV sidecar.
- Idempotência via header Idempotency-Key.
- DB: MySQL fiap_hackathon_upload (Prisma).
- Saída: PUT no S3 raw/{uuid} + SendMessage SQS analysis-requested.

2.4 Processing (porta 3002)

Domínio: Pipeline IA — extração de componentes + classificação de riscos.

- Worker SQS long-poll batch 5 + autoscaling K8s HPA por queue depth.
- Pipeline 2 etapas: Gemini 2.5 Flash (vision) → Groq Llama 3.3 70B (text).
- Guardrails: Zod nas saídas + responseSchema do Gemini + temperature 0.2 + prompt injection defense.
- Fallback automático para MockLlmProvider após 3 retries com exponential backoff.
- DB: MongoDB Atlas fiap_hackathon_processing (Mongoose) — documento por análise.

2.5 Report (porta 3003)

Domínio: Materializar relatório em MySQL e expor consulta REST.

- Consumer SQS analysis-completed/failed.
- Materialização local (cópia ECST) — fica autossuficiente para responder GET /api/reports/{id}.
- DB: MySQL fiap_hackathon_report (Prisma).

2.6 Lambda-auth (serverless)

Domínio: Autenticação JWT HS256 + Lambda authorizer customizado.

- 3 handlers: login, register, authorizer.
- DB: MySQL fiap_hackathon_auth (Prisma) — tabela users.
- JWT HS256 + bcrypt cost 10 + scopes (upload:write, report:read).
- Em produção AWS roda como Lambda nativa atrás de API Gateway HTTP API.

- Em ambiente local roda como Lambda real no LocalStack + API Gateway REST com custom id fiapauth.

2.7 Princípio inviolável

Cada serviço é dono do seu DB. **Nenhum serviço lê o banco do outro.** A integração entre processing e report é feita exclusivamente via Event-Carried State Transfer no SQS (payload completo no analysis.completed).

3. Inteligência Artificial (IADT)

3.1 Abordagem escolhida

Combinamos **duas das abordagens do PDF**: detecção de componentes arquiteturais em imagens (Gemini Vision) + uso de LLM para geração de relatório técnico estruturado com guardrails (Groq Llama 3.3 70B). O pipeline é dividido em duas etapas independentes para isolar falhas e permitir uso de modelos especializados em cada tarefa.

3.2 Pipeline em duas etapas

Etapa	Modelo	Entrada	Saída
1. Detecção (vision)	Gemini 2.5 Flash	Imagem do diagrama (bytes)	Componentes + conexões + confidence
2. Classificação (text)	Groq Llama 3.3 70B	JSON dos componentes da etapa 1	Riscos + recomendações + summary

3.3 Modelos e provedores

Provedor	Modelo	Justificativa
Google Gemini	gemini-2.5-flash	Multimodal nativo (vision), free tier generoso, responseSchema estruturado
Groq	llama-3.3-70b-versatile	Inference rápida (>500 tok/s), free tier robusto, suporte a JSON mode
Mock (fallback)	MockLlmProvider	Saída determinística — usada em testes e como fallback de degradação

3.4 Guardrails (controle de entrada, saída e mitigação de alucinações)

- **Schema-locked output:** Gemini chama com responseType=application/json + responseSchema tipado (Type.OBJECT).
- **Validação Zod:** toda saída de LLM passa por ComponentsExtractionSchema.parse() e RisksClassificationSchema.parse() — Zod barra antes de persistir.
- **Temperature 0.2:** baixa criatividade → respostas reproduzíveis e auditáveis.
- **maxOutputTokens 16384:** evita truncamento (problema do thoughtsTokenCount no Gemini).
- **System prompts restritivos:** instrui o modelo a recusar entradas suspeitas (defesa contra prompt injection).
- **extractionConfidence / classificationConfidence:** modelo declara confiança própria; abaixo do threshold marca degraded.

3.5 Tratamento de falhas da IA

- Cada chamada Gemini/Groq tem **3 retries com exponential backoff** (1s → 2s → 4s).
- Se mesmo após retries o provider falhar, troca para **MockLlmProvider** e marca degraded=true.
- Falhas não-retentáveis (Zod inválido, S3 GET 404) vão para analysis.failed com reason enumerado.
- Após 3 falhas no SQS, mensagem vai para DLQ + alerta Datadog (oncall).

3.6 Demonstração prática

Pipeline foi validado ponta-a-ponta com diagramas reais (incluindo o próprio diagrama de containers deste projeto). Resposta típica em ~25–30s com providerChain={step1:gemini, step2:groq} e degraded=false. A collection Postman em postman/fiap-hackathon.postman_collection.json automatiza a sequência register → login → upload → poll /reports/{id}.

3.7 Limitações reconhecidas

- Free tier Gemini = 50 req/min — rate limit pode ativar o fallback durante demo.
- Modelo enxerga PNG/JPG, mas ainda erra em diagramas muito densos (50+ componentes).
- Não distingue confiavelmente camadas de rede (subnet, AZ) — limitação visual.
- extractionConfidence é auto-reportada — não é métrica calibrada estatisticamente.
- Saída em inglês (idioma do prompt) — multi-idioma fica como stretch goal.

4. Integração IA + Sistema

4.1 Como a IA é acionada

A IA **não é um script isolado**. Ela é acionada automaticamente quando o processing consome uma mensagem de analysis-requested (SQS long-poll). Sequência: o serviço baixa o arquivo do S3, passa pelo LlmPipeline (Strategy Pattern com provider selecionável via env), e publica analysis-completed com payload completo. O cliente nunca chama a IA diretamente — interage apenas com o BFF (REST).

4.2 Como o sistema trata falhas da IA

- Provider lança LlmProviderError(message, providerName, cause, retryable).
- LlmPipeline.withRetry() respeita o flag retryable + 3 tentativas + backoff.
- Esgotadas as tentativas, fallback para Mock — publica analysis.completed com degraded=true.
- Erros não-retentáveis (Zod, schema) → analysis.failed com reason enumerado: DOWNLOAD_FAILED, SCHEMA_VALIDATION_FAILED, AI_PROVIDER_EXHAUSTED, TIMEOUT, INTERNAL.
- Alerta Datadog dispara quando mensagem cai em DLQ.

4.3 Como o resultado da IA é persistido

MongoDB Atlas via Mongoose — coleção analysis_results. Cada documento contém o estado completo: status, providerChain (step1/step2), degraded, durationMs, components, risks, recommendations, errorReason. A escolha por Mongo é justificada pela estrutura semi-estruturada do output da IA (campos variáveis por análise) e ausência de joins com outras entidades.

4.4 Como o relatório é gerado a partir da análise

Após persistir em Mongo, o processing publica analysis.completed no SQS com o **payload completo** (Event-Carried State Transfer). O serviço report consome o evento e materializa uma cópia local em MySQL via Prisma. A consulta GET /api/reports/{id} responde com essa materialização — sem precisar consultar Mongo. Princípio: report fica autossuficiente para responder consultas, mesmo se Mongo estiver fora.

5. Tecnologias Utilizadas

5.1 Backend

Tecnologia	Versão	Justificativa
NestJS	11	Framework enterprise-ready, DI nativa, modularização, decorators TS
TypeScript	5.x	Tipagem estática, melhor DX, redução de bugs em runtime
Prisma	6	ORM type-safe com migrations — usado em upload, report e lambda-auth
Mongoose	8.x	ODM maduro para MongoDB com schemas e validação — usado em processing
Zod	3.x	Validação de schema runtime — guardrail das saídas IA e DTOs HTTP
Pino	9.x	Logger estruturado JSON, alto throughput, PII redact via paths

5.2 Bancos de dados (polyglot persistence)

Banco	Uso	Justificativa
MySQL 8 (RDS)	upload, report, auth	Relacional, ACID, integridade referencial, maturidade
MongoDB Atlas	processing	Documento semi-estruturado (output IA varia), free tier M0

5.3 Mensageria

Componente	Função	Justificativa
AWS SQS	Filas (3x) + DLQ (2x)	Garantia de entrega, retry, DLQ, integração nativa K8s/Lambda
AWS SNS	Topic analysis-events	Fan-out se precisar adicionar consumidores futuros
LocalStack 3.7	Emulação local	Subir SQS/SNS/S3/Lambda/IAM em dev sem custo AWS

5.4 IA / LLM

Provedor	Modelo	Função
Google Gemini	gemini-2.5-flash	Vision — extração de componentes
Groq	llama-3.3-70b-versatile	Text — classificação de riscos e recomendações
Mock (in-process)	—	Fallback determinístico em caso de falha permanente

5.5 Autenticação

Componente	Função	Justificativa
AWS Lambda	Authorizer + login + register	Serverless, escala automática, custo zero quando idle
API Gateway HTTP API	Roteamento + JWT auth	Integração nativa com Lambda, throttling, CORS

Componente	Função	Justificativa
JWT HS256	Tokens stateless	Padrão da indústria, scopes em claims
bcrypt cost 10	Hash de senha	Resistente a brute-force, padrão OWASP

5.6 Infraestrutura

Tecnologia	Função	Justificativa
AWS EKS	Orquestração (target prod)	K8s gerenciado, integração com IAM/IRSA
Terraform 1.9	IaC	Versionamento, plan/apply, state remoto
Helm 3	Deploy K8s	Chart unificado para os 4 NestJS
Docker (multi-stage Alpine)	Containers	Imagens enxutas (~150 MB), non-root, dumb-init
GitHub Container Registry	Registry imagens	Multi-arch amd64+arm64, free, integração GH Actions

5.7 Observabilidade

Ferramenta	Função	Justificativa
Pino	Logs estruturados JSON	Alto throughput, PII redact, request-id
Datadog APM	Tracing distribuído + métricas	Free trial, dashboards prontos, alertas DLQ
Healthchecks NestJS	/api/health/live + /api/health/ready	Compatível com K8s probes

5.8 Qualidade

Ferramenta	Função	Cobertura
Jest 29	Testes unitários	108 testes verdes, gate 80% em todos os 5 services
Supertest + Jest	E2E	Smoke ponta-a-ponta no infra (Postman + jest E2E)
ESLint + Prettier	Lint/format	Configuração centralizada, pre-commit hook

6. Infraestrutura e DevOps

6.1 Docker

Todos os 4 microsserviços NestJS são containerizados com **Dockerfile multi-stage Alpine** — stage builder com devDeps + tsc, stage runtime apenas com dist/ + node_modules de produção. Imagens finais entre 150–200 MB. Container roda como usuário não-root (UID 1001), entrypoint via dumb-init para forwarding de signals e reaping de zumbis.

6.2 Docker Compose

Dois composes em fiap-hackathon-infra/e2e/:

- docker-compose.e2e.yml — build local dos 4 NestJS + MySQL + Mongo + LocalStack + ClamAV.

Usado em dev e como base do CI e2e-smoke.

- docker-compose.e2e.ghcr.yml — override que troca build: local por image:

ghcr.io/danielroberto72/...:latest. Permite ao avaliador rodar make full-ghcr clonando apenas o repo do infra.

- docker-compose.real-llm.yml — override que troca providers de mock para Gemini + Groq reais (requer GEMINI_API_KEY + GROQ_API_KEY exportadas).

6.3 Pipeline CI/CD

GitHub Actions em cada um dos 6 repos. Padrão para os 4 NestJS:

- **Job test** (todo push + PR): npm ci → prisma generate → npm test → npm run build. Coverage gate 80%.
- **Job build-and-push** (push em main): docker buildx multi-arch (linux/amd64 + linux/arm64) com QEMU + push GHCR com tags :latest, :sha-{short}, :main.
- Cache de build via type=gha — builds incrementais em segundos.
- Permissions mínimos: contents:read, packages:write + secrets.GITHUB_TOKEN.

Para o lambda-auth: npm test → tsc + tsc-alias → zip → upload-artifact + softprops/action-gh-release@v2 promove o zip para Release rolling release-latest.

Para o infra: terraform fmt/validate → helm lint/template → e2e-smoke (pull imagens GHCR → docker compose up → npm test → tear down). Roda em push, PR e schedule diário 06:00 UTC.

6.4 Deploy local via GHCR — caminho de demonstração

O PDF aceita explicitamente *Deploy local OU cloud*. Decidimos por **deploy local via GHCR** como caminho principal de demo (decisão registrada na ADR-002 D17). Avaliador roda o sistema completo com um único comando, clonando apenas o repo do infra:

```
git clone https://github.com/DanielRoberto72/fiap-hackathon-infra
cd fiap-hackathon-infra/e2e
make full-ghcr # pull imagens GHCR + docker-compose up + jest E2E + tear down
```

6.5 Lambda local via LocalStack

O lambda-auth roda como **Lambda real no LocalStack Community** com API Gateway REST API v1 (custom id determinístico fiapauth). O bootstrap automatizado em localstack-init/02-bootstrap-auth.sh baixa o zip do GitHub Release, cria role IAM, 3 funções Lambda (login, register, authorizer) e wireia routes POST /auth/login + POST /auth/register com Lambda Proxy Integration. Resultado: autenticação funciona serverless de verdade mesmo em ambiente local.

6.6 Capacidade de deploy AWS (provada, não armada)

A stack completa em AWS (EKS + RDS + S3 + SQS + Lambda + API Gateway + IRSA) está pronta como código em terraform/ e helm/. O CI valida via terraform fmt/validate e helm lint/template a cada PR. **Não foi aplicada** em conta AWS para esta entrega (decisão estratégica D16 + D17 — economia de custo + tempo). Armar é uma flag `AWS_DEPLOY_ENABLED=true` de distância.

7. Qualidade e Observabilidade

7.1 Testes (108 verdes, cobertura 80%+ em todos)

Serviço	Testes	Coverage
bff	22	100% statements / 100% branches / 100% funcs
upload-orchestration	28	99% statements / 95%+ branches
processing	11	100% statements
report	15	100% statements / 81% branches
lambda-auth	30	98% statements / 86% branches
E2E (infra)	2	Happy path + idempotência
Auth E2E (LocalStack)	3	register + login + JWT
TOTAL	108+ verdes	Gate 80% em todos os jest.config

7.2 Logs estruturados

Todos os serviços usam **Pino** com saída JSON. Cada log carrega requestId, analysisId, userId, e contexto do bounded context. **PII redact** configurado em logger.redact.paths remove password, req.headers.authorization e *.accessToken.

7.3 Tratamento de erros

- Erros de domínio estendem AuthError/UploadError/AnalysisError com httpStatus e code enumerados.
- Filter NestJS global converte para envelope JSON consistente {error: {code, message}}.
- Pipeline IA: erros classificados em retryable: true|false — controla decisão de retry.
- Razão de falha persistida em analysis_results.errorReason + errorDetail para auditoria.

7.4 Observabilidade em produção

- Datadog APM via dd-trace com auto-instrumentação HTTP/Mongo/Prisma/SQS.
- Métricas custom: llm.duration_ms, llm.degraded.count, sqs.dlq.depth.
- Alerta DLQ: monitor dispara quando depth > 0 por 5 min.
- Dashboards prontos em terraform/datadog.tf (provider Datadog) — Helm não aplica em local.

7.5 Documentação

- **README mestre** consolidado em fiap-hackathon-infra/README.md com 14 seções.
- **ADR-002** com 18 decisões arquiteturais documentadas (D1 a D18).
- **3 diagramas Eraser**: containers, sequence happy path, fluxo de falha IA.
- **docs/seguranca.md** com 10 seções obrigatórias do hackathon.
- **postman/** com collection v2.1 + DEMO.md (roteiro de gravação).

8. Segurança (seção obrigatória)

8.1 Requisitos básicos adotados

- **TLS 1.2+** obrigatório no API Gateway (rejeição automática de protocolos legacy).
- **JWT HS256** com expiração 1h, scopes em claims, validado por Lambda authorizer em todas as rotas privadas.
- **bcrypt cost 10** no armazenamento de senhas — sem texto puro nem hash trivial.
- **Secrets Manager + IRSA** em produção: nenhuma credencial em env hardcoded ou em ConfigMap.
- **Pino PII redact** remove campos sensíveis antes de logs serem persistidos.

8.2 Validação e tratamento de entradas não confiáveis

- **Upload**: validação MIME (Content-Type), magic bytes via lib file-type, tamanho máximo (10MB), e **scan ClamAV** antes de persistir no S3.
- **Body JSON**: validação Zod por DTO em todos os endpoints — rejeição com 400 detalhado.
- **Idempotency-Key** obrigatória em POSTs de criação — protege contra retry duplicado do cliente.
- **Rate limit** no API Gateway (10 req/s por IP, 100 req/s burst).
- **CORS allowlist** no API Gateway — sem wildcard.

8.3 Uso controlado da IA (escopo e previsibilidade)

- **Schema-locked output**: Gemini chamado com responseSchema tipado (Type.OBJECT) — não consegue retornar texto livre.
- **Validação Zod** nos JSONs de saída antes de qualquer persistência ou propagação.
- **Temperature 0.2** em ambos os modelos — minimiza criatividade aleatória.
- **System prompts restritivos**: instruem o modelo a recusar pedidos que extrapolem o domínio (análise de diagrama).
- **Defesa contra prompt injection**: prompts do usuário (caption opcional) são sanitizados e wrapeados em delimiter <user_input>...</user_input>.
- **maxOutputTokens 16384**: evita corte de JSON por estouro de orçamento de tokens.

8.4 Tratamento seguro de falhas da IA

- Retry exponencial 3x (1s → 2s → 4s) — não cascadeia falha do provider para o cliente.
- Fallback automático para **MockLlmProvider** com degraded=true — relatório sempre é entregue.
- Falhas não-retentáveis (Zod, schema) → analysis.failed com reason enumerado.
- Após DLQ, alerta Datadog dispara oncall — operador humano revisa.
- extractionConfidence / classificationConfidence auto-reportadas pelo modelo são gravadas no documento para auditoria.

8.5 Comunicação segura entre serviços

- Em produção AWS: **API Gateway** → **VPC Link** → **NLB interno** → **EKS**. Nenhum microsserviço fica exposto publicamente.

- **IRSA (IAM Roles for Service Accounts):** uma role por pod, com privilégio mínimo definido em terraform/iam.tf.
- **Security Groups** restritivos: porta 3000–3003 só acessível dentro da VPC.
- **TLS interno** via service mesh (Linkerd) — stretch goal documentado em ADR-002.
- Em LocalStack local: rede Docker isolada fiap-hackathon-e2e_e2e + DNS interno.

8.6 Hardening do container

- Imagem base Alpine 3.20 (~30 MB) — superfície de ataque mínima.
- Container roda como usuário não-root (UID 1001).
- dumb-init como entrypoint — sem shell exposto em PID 1.
- Filesystem read-only exceto /tmp (configurado no K8s securityContext).
- capabilities: drop ALL + privileged: false.

8.7 Riscos identificados, mitigações atuais e stretch goals

Risco	Mitigação atual	Stretch goal
MongoDB Atlas aberto a 0.0.0.0/0	Usuário/senha forte + IP allowlist do EKS NAT	VPC peering Atlas ↔ AWS VPC
Falta de WAF no API Gateway	Validação Zod + rate limit nativo	AWS WAF v2 com managed rule sets
External Secrets Operator não implementado	Secrets do K8s populados manualmente uma vez	ESO sincronizando do Secrets Manager
Sem mTLS entre serviços	Tráfego confinado em VPC privada	Linkerd mesh com mTLS automático
Datadog free trial = janela curta	Trial iniciado próximo à demo (~ 14 dias)	Plano pago ou switch para Grafana/Loki self-hosted
Free tier LLM = rate limit	Fallback automático para Mock + degraded flag	Plano pago Gemini + Groq

9. Considerações de Design

9.1 Consistência eventual

Cada serviço mantém consistência forte em seu banco local. A consistência entre serviços é alcançada via processamento de eventos (SQS analysis-completed). Em caso de falha, o mecanismo de fallback (degraded) garante que o relatório sempre é entregue, ainda que sinalizado para revisão humana.

9.2 Idempotência

- Idempotency-Key obrigatória em POST /api/analyses.
- Verificação de estado no consumer SQS antes de aplicar mudanças.
- Mensagens SQS são processadas com base no analysisId (UUID único).
- Logs detalhados com requestId + analysisId para auditoria.

9.3 Resiliência

- Retentativas SQS via visibilityTimeout + maxReceiveCount 3.
- Dead Letter Queues (2x) para mensagens problemáticas.
- Fallback automático para Mock após exaurir retries do provider IA.
- Healthchecks /api/health/live e /api/health/ready — K8s reinicia pod automaticamente.
- Multi-AZ em produção (EKS managed) — stretch goal: read replica RDS.

10. Conclusão

A arquitetura de microsserviços com comunicação assíncrona via SQS, polyglot persistence, pipeline IA em duas etapas com guardrails Zod e fallback automático, deploy local serverless via GHCR + LocalStack Lambda e cobertura de testes 80%+ proporciona um sistema:

- **Escalável:** cada serviço escala independentemente, fila SQS absorve picos.
- **Resiliente:** sem ponto único de falha, fallback automático, DLQ + alerta.
- **Manutenível:** bounded contexts isolados, ADR documentando todas as decisões.
- **Reprodutível:** avaliador roda make full-ghcr e tem o sistema completo em ~90s.
- **Observável:** Pino structured logs + Datadog APM + métricas custom.
- **Seguro:** validação em múltiplas camadas, guardrails IA, hardening de container.

Esta entrega atende cirurgicamente aos requisitos do Hackathon Integrado IADT+SOAT — funcionalidades obrigatórias (upload + processo de análise + status + relatório), requisitos técnicos (microsserviços + REST + assíncrono + Clean Architecture + DB próprio + testes), IA (pipeline + guardrails + fallback + limitações documentadas), infraestrutura (Docker + Compose + CI/CD com Build + Testes + Deploy local verificado em runner GitHub Actions) e seção obrigatória de Segurança.